

FrogOS

declarative intent -> observed state -> ordered plan -> commit



Кроитор Александр / научный руководитель: Nichita Nartea / 2026

Linux state drifts

Админ описал **desired state**.

- package доступен;
- service включён;
- user состоит в группах;
- профиль реагирует на процесс.

Хост ушёл в **state drift**.

- пакет удалили;
- service упал;
- группа пропала;
- power profile сменился.

Задача FrogOS: не выполнить скрипт, а вернуть систему к intent.

Функциональные требования

Intent path

- принять декларативную конфигурацию;
- проверить типы, семантику и бизнес логику;
- сохранить IR как `intent.json`;
- явно версионировать контракт.

Runtime path

- собрать `SystemState`;
- построить delta между графами;
- применить `ordered plan`;
- отправить `commit` или `rollback`.

Функционально FrogOS должна закрывать полный путь: `describe -> observe -> plan -> execute -> store`.

Нефункциональные требования

Safety

undo для каждого шага,
dry-run путь, daemon
boundary

Determinism

одинаковые входы дают
одинаковый план

Audit

поколение хранит
проверенный intent и
результат применения

Maintainability

- новые домены проходят DSL -> IR -> Toad -> Planner -> Executor;
- side effects локализованы в backend traits.

Portability

- сборка через flakes;
- e2e в FHS-контейнере;
- Rust/Haskell boundary через protobuf.

Reconciliation loop

$$I \times S_t^{obs} \rightarrow P_t \rightarrow S_{t+1}^{obs}$$

I

проверенный intent

Sobs

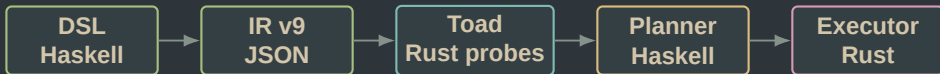
то, что увидел Toad

P

ordered plan + undo

FrogOS не знает абсолютную истину о машине. Каждый цикл начинается с нового наблюдения.

Компоненты не смешивают ответственность



Чистые слои

- DSL validation;
- IR schema version;
- graph delta;
- JSON/protobuf contracts.

Side effects

- Nix package store;
- dinit modules;
- users/groups;
- network rules.

Intent читается как сценарий, но не является скриптом

```
gaming :: ConfigBuilder ()
gaming = profile "gaming" $
  when (processRunning "steam" `via` poll 500) $ do
    disable nginx
    setPowerProfile Performance
```

Condition

когда профиль активен

Action

какое состояние нужно

Policy

какие варианты
допустимы

NixOS фиксирует систему, FrogOS реагирует на состояние

NixOS

```
{ pkgs, ... }: {  
  services.nginx.enable = true;  
  users.users.alice = {  
    isNormalUser = true;  
    extraGroups = [ "wheel" "docker" ];  
  };  
  environment.systemPackages = [  
    pkgs.dust  
  ];  
}
```

Фичи: reproducible rebuild, generations, Nix store.

FrogOS

```
profile "desktop" $ do  
  enable nginx  
  user "alice" $ do  
    normalUser True  
    extraGroups [ "wheel", "docker" ]  
    exposePackage "dust"  
  
profile "gaming" $  
  when (processRunning "steam" `via` poll 500) $  
    disable nginx
```

Фичи: runtime observation, reactive profiles, delta plan.

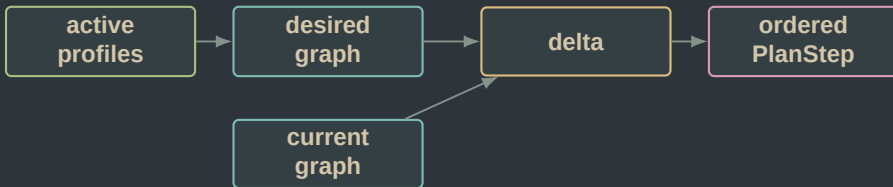
FrogOS не заменяет Nix. Он использует Nix как backend там, где нужны пакеты и store.

JSON-контракт переживает runtime boundaries

```
{
  "version": 9,
  "condition": {
    "type": "process_running",
    "name": "steam",
    "observe": { "type": "poll", "interval_ms": 500 }
  },
  "actions": [
    { "type": "module_disable", "priority": 100 },
    { "type": "power_profile", "value": "performance", "priority": 100 }
  ]
}
```

Версия схемы даёт Planner право явно отвергнуть несовместимый intent.

Planner считает delta, а не команды из профиля



Invariant

- equal inputs -> equal plan;
- reconciled resource -> no step;
- conflict -> planning error.

Ordering

- groups before users;
- users before memberships;
- packages before modules.

Почему нужен граф ресурсов

Desired graph

- строится из IR и активных профилей;
- priority разрешает конкурирующие желания;
- effect resources не становятся состоянием сами по себе.

Current graph

- строится из SystemState;
- содержит только наблюдаемые факты;
- delta удаляет уже согласованные ресурсы.

Plan = minimal difference + backend-safe ordering + undo metadata

PlanStep несёт действие и обратный шаг

```
#[serde(tag = "type", rename_all = "snake_case")]
pub enum DeltaAction {
    GroupAdd { name: String },
    UserAdd { config: UserConfig },
    MembershipAdd { user: String, group: String },
    PackageInstall { name: String },
    ModuleEnable { module: ModuleRef, priority: u32 },
    NetworkApply { allow_ports: Vec<u16> },
}
```

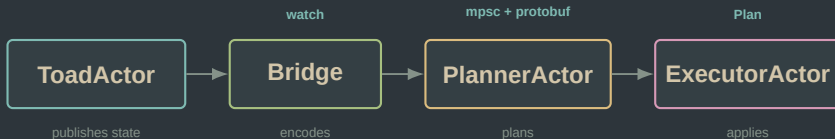
DeltaAction = данные. Side effect = только dispatch в Executor.

Реальный ordered plan заканчивается commit

```
[1/11] add group docker
[2/11] add group networkmanager
[3/11] add group nobody
[4/11] add group nogroup
[5/11] add group wheel
[6/11] add user yorunikakeru
[7/11] add yorunikakeru -> group docker
[8/11] add yorunikakeru -> group networkmanager
[9/11] add yorunikakeru -> group wheel
[10/11] install package dust
frogosd::supervisor_runtime: store commit
```

Лог показывает не “успешную команду”, а ordered reconciliation plan.

Supervisor держит actor pipeline живым



rest-for-one

- Toad -> вся downstream chain;
- Bridge -> Planner + Executor;
- Executor -> новая execution chain.

Почему так

- paired ends у каналов;
- state source должен быть цельным;
- restart rate limited.

Что защищает архитектура

Correctness

типы DSL, IR v9,
golden/round-trip tests

Recovery

undo в каждом PlanStep,
rollback в обратном
порядке

Isolation

FHS e2e, daemon socket,
generation store

**Итог: self-healing как
reconciliation loop.**

- intent проверяется;
- state наблюдается;
- delta применяется с undo.